LAPORAN PRAKTIKUM

Teknologi IoT II

Modul 7: Integrasi Backend Server Lokal (Docker & Mosquitto) dan Rule Engine (Node-RED)

DISUSUN OLEH:

Nama : Zahara Ummu Abdillah

NPM : 25782058

Kelas :3B

Dosen Pengampu:

Dr. Ir. Septafiansyah Dwi Putra

Agus Ambarwari, S.Pd., M.kom

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026/2027

BAB 7.3: TEORI DASAR

- **Ringkasan Konsep Utama:**
    - **Edge/Fog Computing:** Menempatkan server pemroses di gerbang lokal (*Gateway*) untuk mengurangi ketergantungan pada *cloud*.
    - **Docker & Docker Compose:** Teknologi kontainerisasi ringan yang membungkus aplikasi beserta dependensinya agar berjalan konsisten di atas *Host OS*.
    - **Eclipse Mosquitto:** Broker MQTT ringan berbasis C untuk komunikasi berbasis protokol TCP (port 1883) dan WebSockets (port 9001).
    - **Node-RED:** Perangkat lunak pemrograman visual berbasis aliran (*Flow-Based Programming*) menggunakan objek paket data bernama msg.
    - **Rule Engine & Closed-Loop:** Sistem pengambil keputusan otomatis (misal: *Bang-Bang Control*) yang membandingkan variabel sensor terhadap *setpoint* untuk menggerakkan aktuator.
    - **Keamanan MQTT:** Sistem proteksi dengan menonaktifkan akses anonim (allow_anonymous false) dan mewajibkan verifikasi kredensial *hash* sandi.

7.4 Persiapan Praktikum

7.4.1 Kebutuhan Perangkat Keras

Pastikan komponen berikut tersedia di meja praktikum Anda (identik dengan perangkat praktikum Modul 6):

1. *Board* NodeMCU ESP8266 (1 unit)
2. Kabel Data Micro-USB (1 unit)
3. *Breadboard* / Papan Proyek (1 unit)
4. Sensor Suhu & Kelembapan DHT11 atau DHT22 (1 unit)
5. LED Indikator 5mm (1 unit)
6. Resistor 220 Ω (1 unit, pembatas arus LED)
7. Kabel *Jumper* tipe *Male-to-Male* dan *Male-to-Female* (secukupnya)

8. Komputer/Laptop pengembang (Windows, macOS, atau Linux) dengan RAM minimal 4 GB.
9. Sumber Jaringan Wi-Fi Lokal (Disarankan menggunakan *Tethering / Mobile Hotspot* dari ponsel di mana laptop dan NodeMCU terhubung ke subnet yang sama).

7.4.2 Persiapan Perangkat Lunak & Lingkungan Docker

1. **Docker Engine & Docker Compose:**
    - Untuk pengguna **Windows / macOS**: Unduh dan pasang **Docker Desktop** dari situs resmi docker.com. Pastikan status Docker Desktop di pojok bilah tugas menunjukkan ikon hijau bertuliskan **Engine Running**.
    - Untuk pengguna **Linux**: Pastikan paket docker dan plugin docker-compose-plugin telah terpasang dan servis Docker aktif (sudo systemctl start docker).
2. **Arduino IDE:**
    - Pastikan pustaka berikut telah terpasang melalui *Library Manager*:
        - PubSubClient oleh Nick O'Leary.
        - ArduinoJson oleh Benoit Blanchon (versi 6.x disarankan).
        - DHT sensor library oleh Adafruit beserta dependensi *Adafruit Unified Sensor*.
3. **Aplikasi Teks Editor:**
    - Visual Studio Code, Notepad++, atau editor teks standar untuk membuat berkas docker-compose.yml dan mosquitto.conf.
4. **Peramban Web Modern:**
    - Google Chrome, Mozilla Firefox, atau Microsoft Edge untuk mengakses dasbor Node-RED di port 1880.

7.5 Praktikum

7.5.1 Praktikum 1: Pembangunan Ekosistem IoT Server dengan Docker Compose

```
D:\DOCKER\IoT_Server2>Docker compose up -d
[+] up 29/29
 ⠿ Image nodered/node-red:latest  Pulled                                    179.4s
 ⠿ Image eclipse-mosquitto:latest Pulled                                     94.3s
 ⠿ Network iot_server2_default    Created                                     0.2s
 ⠿ Container mqtt-broker          Started                                     2.7s
 ⠿ Container node-red             Started                                     1.9s

D:\DOCKER\IoT_Server2>
```

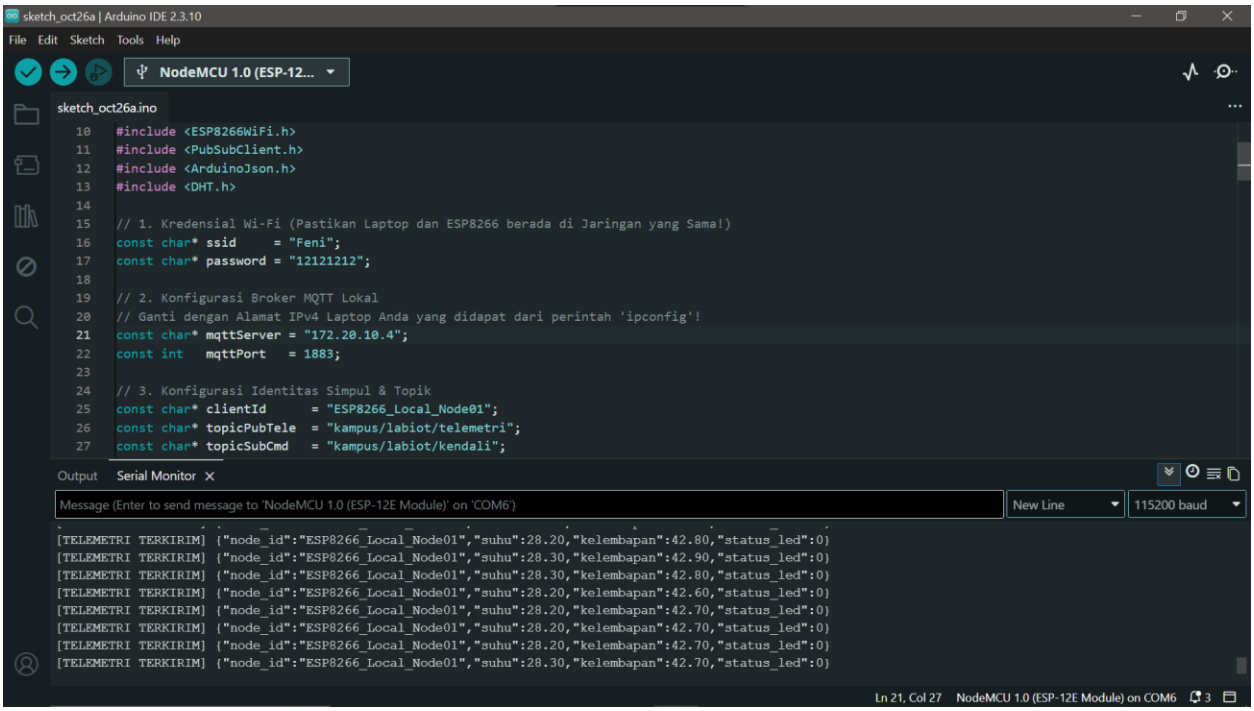
```
D:\DOCKER\IoT_Server2>docker ps
CONTAINER ID   IMAGE                    COMMAND                  CREATED          STATUS                    PORTS
                                                                                 NAMES
b06d8cb2fc33   nodered/node-red:latest  "./entrypoint.sh"        31 minutes ago   Up 31 minutes (healthy)   0.0.0.0:18
80->1880/tcp, [::]:1880->1880/tcp                                                node-red
a667ad8381aa   eclipse-mosquitto:latest "/docker-entrypoint.…"   31 minutes ago   Up 31 minutes             0.0.0.0:18
83->1883/tcp, [::]:1883->1883/tcp, 0.0.0.0:9001->9001/tcp, [::]:9001->9001/tcp   mqtt-broker

D:\DOCKER\IoT_Server2>
```

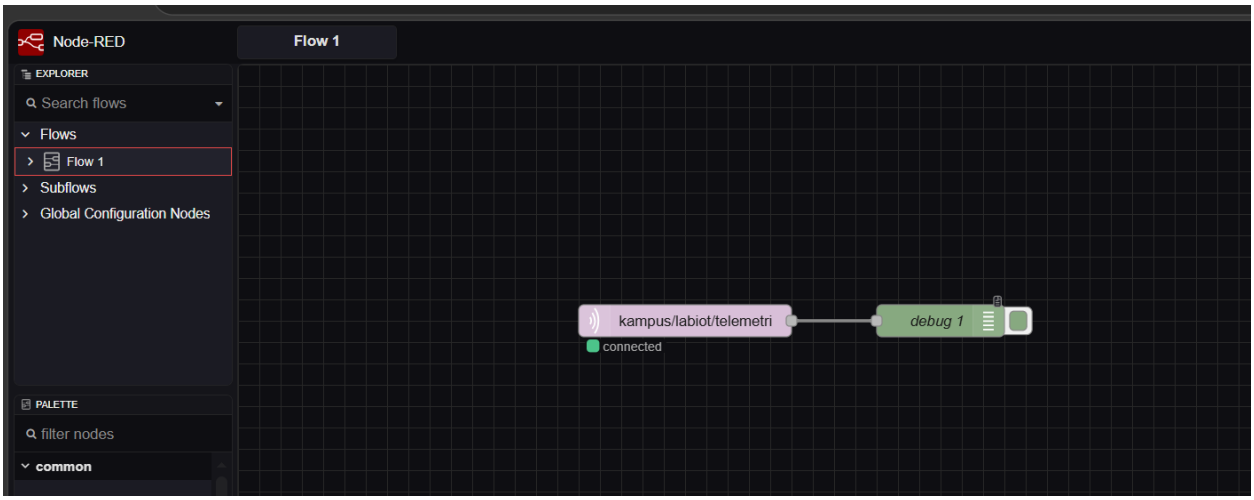
7.5.2 Praktikum 2: Migrasi Simpul NodeMCU ESP8266 ke Broker MQTT Lokal

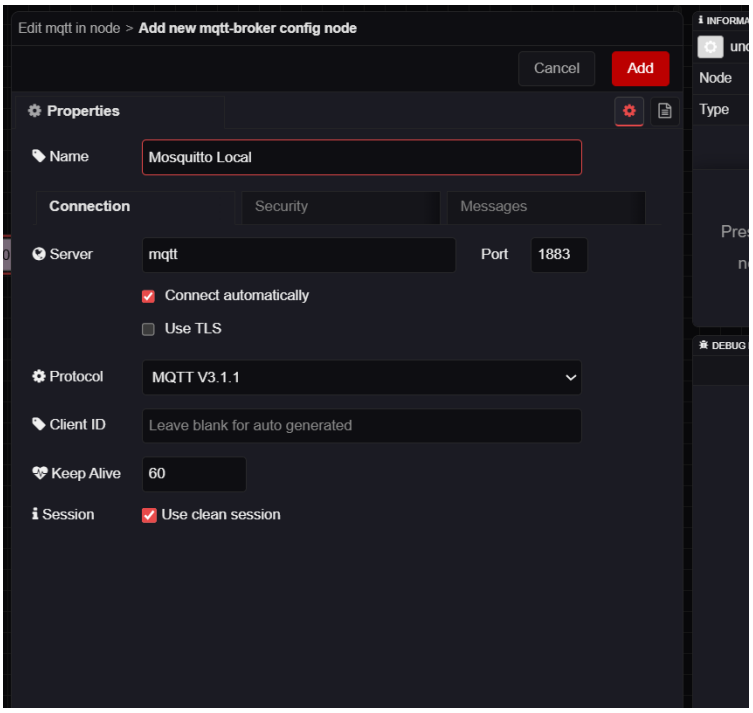
```
Wireless LAN adapter Wi-Fi:

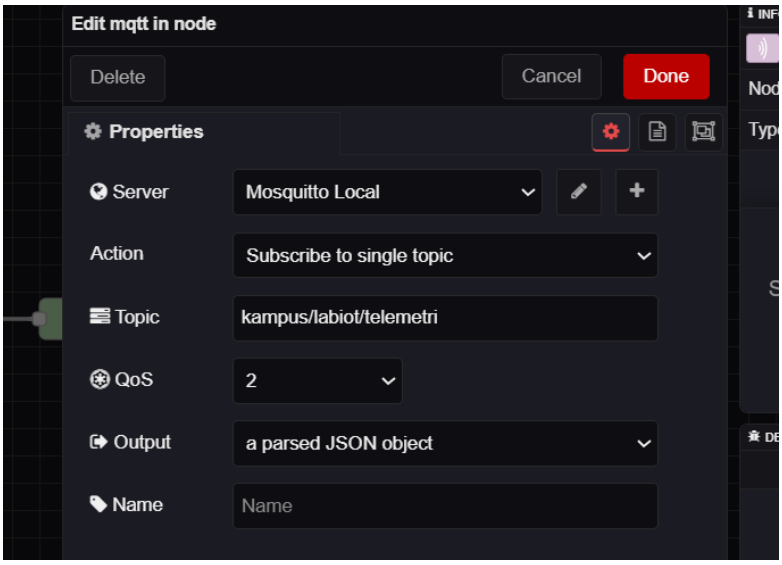
   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::ae6f:2f46:8409:fc04%8
   IPv4 Address. . . . . . . . . . . : 172.20.10.4
   Subnet Mask . . . . . . . . . . . : 255.255.255.240
   Default Gateway . . . . . . . . . : 172.20.10.1
```



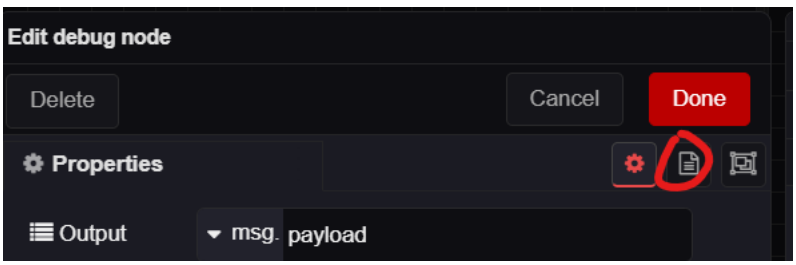
7.5.3 Praktikum 3: Integrasi Alur Data Telemetri Dua Arah pada Node-RED



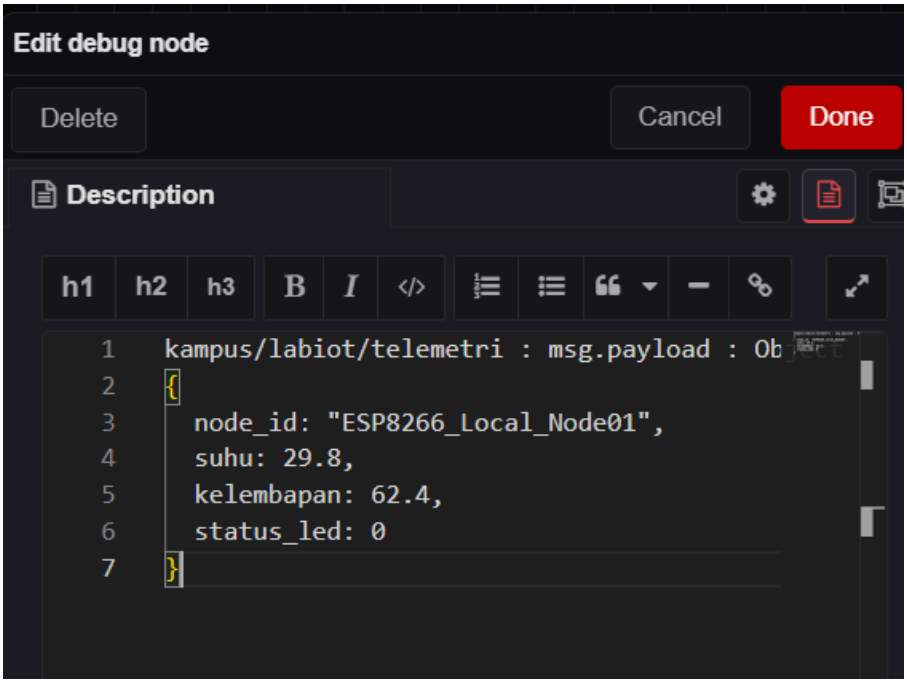


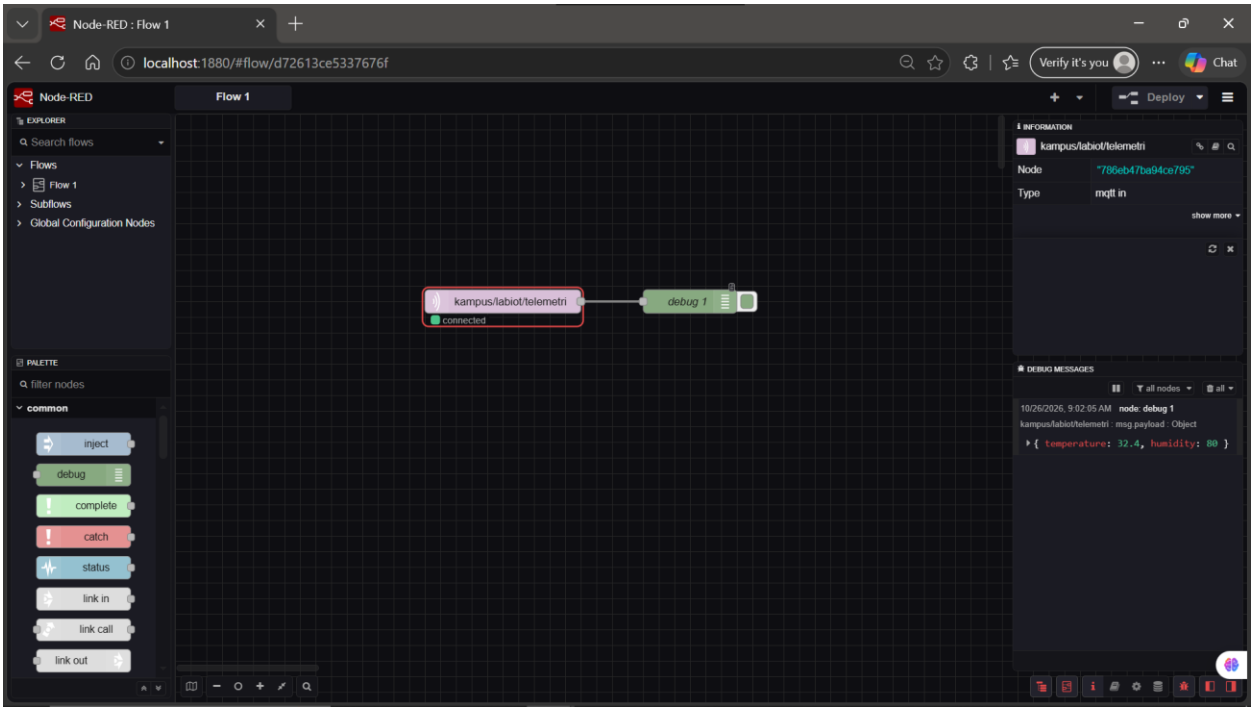


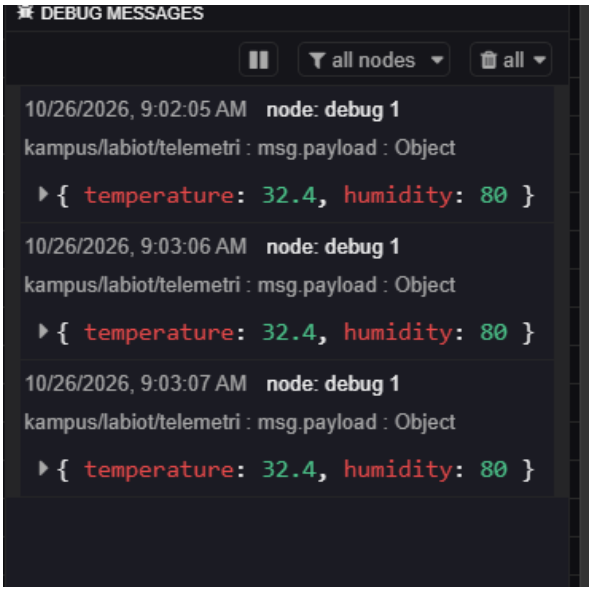
Setelah itu klik 2x debug nya dan pilih note seperti di bawah ini



isi dengan kode seperti ini



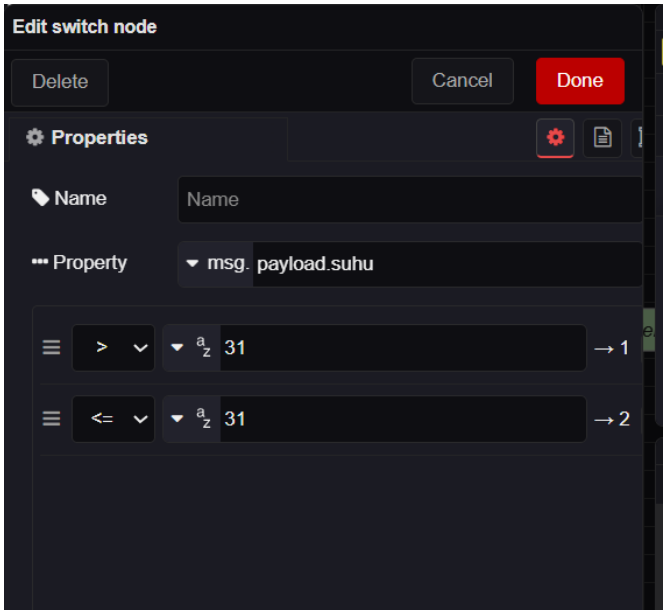




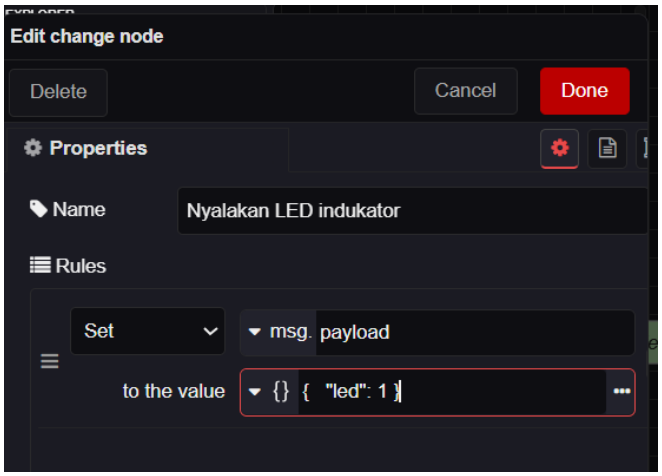
7.5.4 Praktikum 4: Implementasi Rule Engine Otomasi Aktuator Loop Tertutup

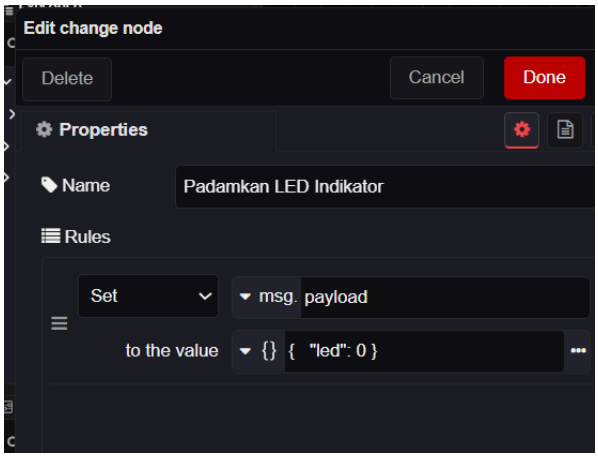
Langkah Kerja:

1. **Membuat Node Percabangan Logika (switch):**
    - Pada kategori function, seret node switch ke kanvas.
    - Hubungkan port keluaran node mqtt in ke port masukan node switch.
    - Klik ganda node switch untuk menyetel aturan logika evaluasi:
        - Property: Ganti msg.payload menjadi msg.payload.suhu.
        - Aturan 1 (Kondisi Panas): Pilih operator >, lalu masukkan angka: 31.
        - Klik tombol + add di bagian bawah untuk menambahkan aturan kedua.
        - Aturan 2 (Kondisi Aman): Pilih operator <=, lalu masukkan angka: 31.
        - Klik Done. Perhatikan bahwa node switch kini memiliki dua port keluaran bernomor 1 (atas) dan 2 (bawah).

- 

2. **Membuat Node Transformasi Muatan Panas (change):**
    - Seret node change dari kategori *function* ke kanvas di sebelah kanan atas node switch.
    - Hubungkan port keluaran 1 (suhu >31) dari node switch ke masukan node change pertama ini.
    - Klik ganda node change:
        - Name: Nyalakan LED Indikator
        - Set: msg.payload
        - To: Ubah tipe data dari string (az) menjadi JSON ({ }), lalu masukkan nilai: { "led": 1 }
        - Klik Done.

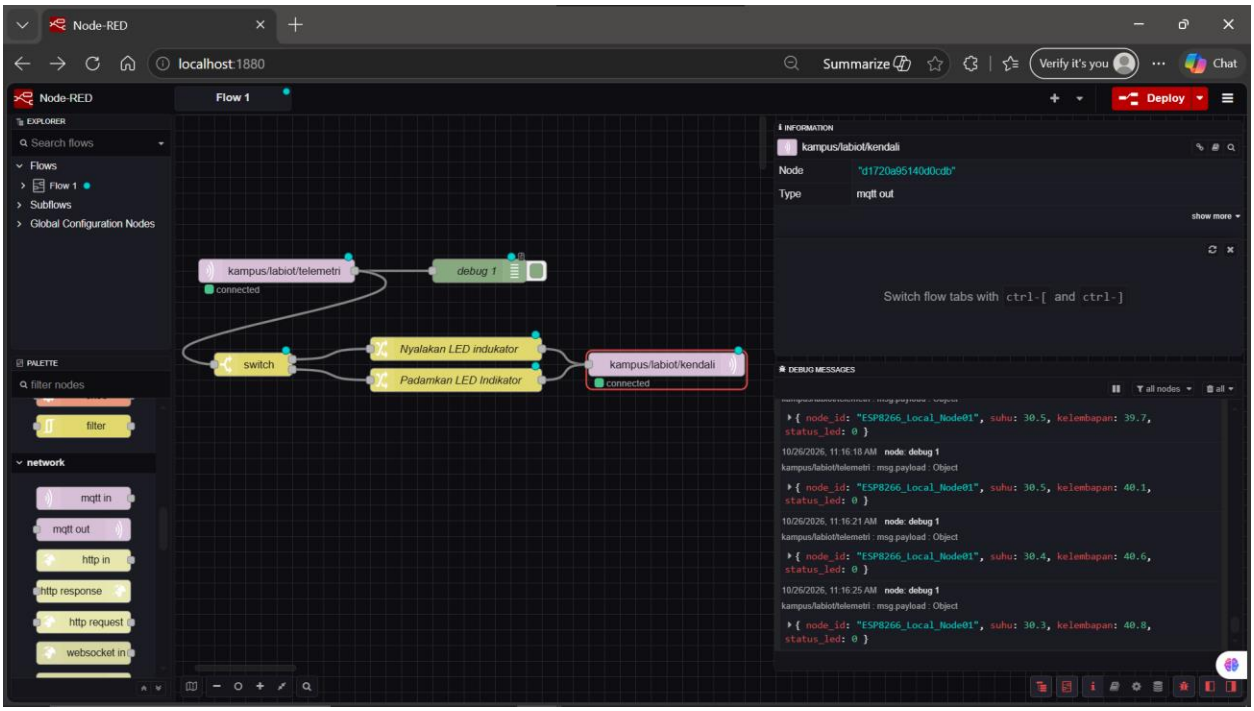
- 

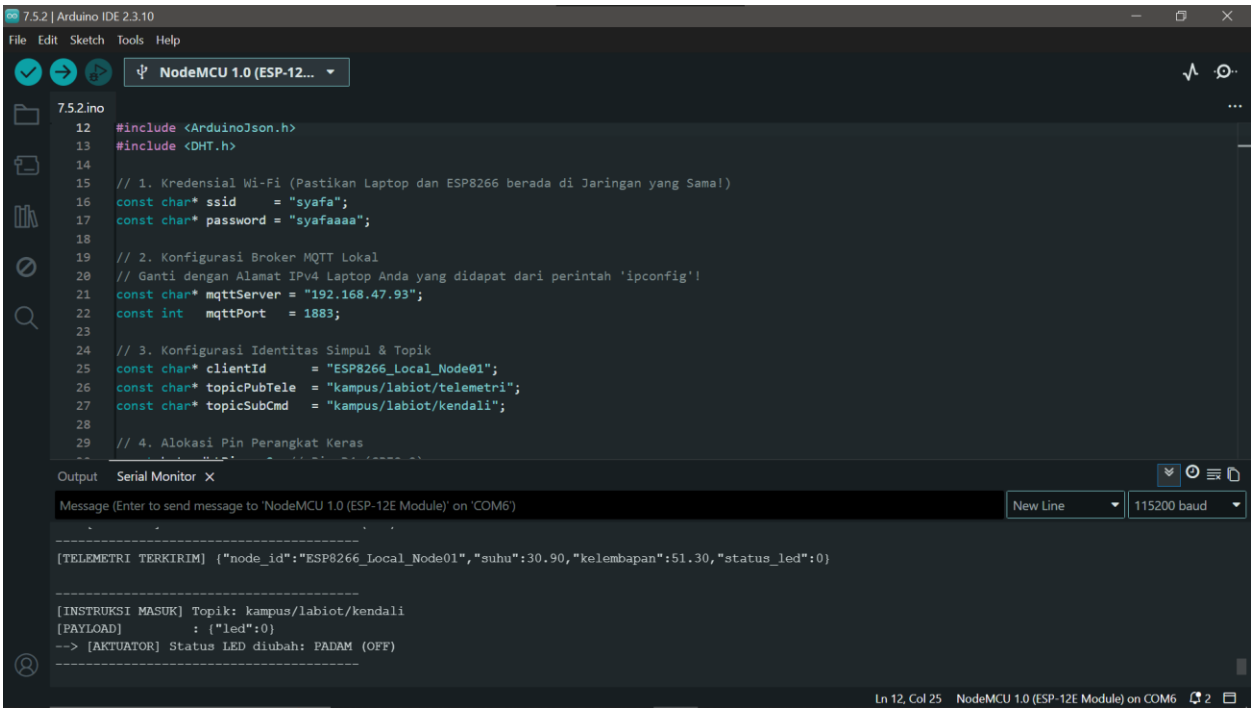
3. **Membuat Node Transformasi Muatan Normal (change):**
    - Seret node change kedua ke kanvas di sebelah kanan bawah node switch.
    - Hubungkan port keluaran 2 (suhu ≤31) dari node switch ke masukan node change kedua ini.
    - Klik ganda node change:
        - Name: Padamkan LED Indikator
        - Set: msg.payload
        - To: Ubah tipe data menjadi JSON ({ }), lalu masukkan nilai: { "led": 0 }
        - Klik Done.
        - 

4. **Menerbitkan Instruksi Balik ke Broker (mqtt out):**

- Pada kategori network, seret node mqtt out ke kanvas.
- Hubungkan keluaran dari node change pertama dan node change kedua secara bersamaan ke port masukan node mqtt out.
- Klik ganda node mqtt out:
  - Server: Pilih Mosquitto Local.
  - Topic: kampus/labiot/kendali
  - QoS: 0 (atau 1)
  - Retain: false
  - **Klik Done.**

5. **Menerapkan Perubahan (*Deploy*):**
   - Klik tombol merah Deploy di kanan atas antarmuka.
6. **Pengujian Fisik Sistem Otomasi Tertutup:**
   - Amati status awal: Karena suhu ruangan umumnya berkisar antara 26∘C−29∘C (≤31∘C), LED indikator berada dalam kondisi padam.
   - Dekatkan sumber panas ringan (seperti ujung jari Anda yang ditempelkan kuat pada sensor DHT, atau hembusan napas hangat) hingga pembacaan suhu pada panel Debug merangkak naik melewati 31.0∘C.
   - Hasil Pengamatan: Seketika suhu melampaui 31.0∘C, Node-RED mengeksekusi port 1, mengirimkan payload {"led": 1}, dan LED indikator bahaya di pin D1 seketika menyala terang!
   - Lepaskan jari Anda dan biarkan suhu mendingin kembali ke bawah 31.0∘C. LED indikator akan padam kembali secara otomatis.


Node-RED
localhost:1880
Flow 1
Deploy
kampus/labiot/telemetri
connected
debug 1
switch
Nyalakan LED indukator
Padamkan LED Indikator
kampus/labiot/kendali
connected
kampus/labiot/kendali
Node
"d1720a95140d0cdb"
Type
mqtt out
Switch flow tabs with ctrl-[and ctrl-]
DEBUG MESSAGES
{ node_id: "ESP8266_Local_Node01", suhu: 30.5, kelembapan: 39.7, status_led: 0 }
{ node_id: "ESP8266_Local_Node01", suhu: 30.5, kelembapan: 40.1, status_led: 0 }
{ node_id: "ESP8266_Local_Node01", suhu: 30.4, kelembapan: 40.6, status_led: 0 }
{ node_id: "ESP8266_Local_Node01", suhu: 30.3, kelembapan: 40.8, status_led: 0 }


7.5.2 | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...
7.5.2.ino
#include <ArduinoJson.h>
#include <DHT.h>
// 1. Kredensial Wi-Fi (Pastikan Laptop dan ESP8266 berada di Jaringan yang Sama!)
const char* ssid = "syafa";
const char* password = "syafaaaa";
// 2. Konfigurasi Broker MQTT Lokal
// Ganti dengan Alamat IPv4 Laptop Anda yang didapat dari perintah 'ipconfig'!
const char* mqttServer = "192.168.47.93";
const int mqttPort = 1883;
// 3. Konfigurasi Identitas Simpul & Topik
const char* clientId = "ESP8266_Local_Node01";
const char* topicPubTele = "kampus/labiot/telemetri";
const char* topicSubCmd = "kampus/labiot/kendali";
// 4. Alokasi Pin Perangkat Keras
Output Serial Monitor
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM6')
New Line
115200 baud
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":30.90,"kelembapan":51.30,"status_led":0}
[INSTRUKSI MASUK] Topik: kampus/labiot/kendali
[PAYLOAD] : {"led":0}
--> [AKTUATOR] Status LED diubah: PADAM (OFF)
Ln 12, Col 25 NodeMCU 1.0 (ESP-12E Module) on COM6

7.5.2 | Arduino IDE 2.3.10

File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12...

7.5.2.ino

```
#include <ArduinoJson.h>
#include <DHT.h>

// 1. Kredensial Wi-Fi (Pastikan Laptop dan ESP8266 berada di Jaringan yang Sama!)
const char* ssid     = "syafa";
const char* password = "syafaaaa";

// 2. Konfigurasi Broker MQTT Lokal
// Ganti dengan Alamat IPv4 Laptop Anda yang didapat dari perintah 'ipconfig'!
const char* mqttServer = "192.168.47.93";
const int   mqttPort   = 1883;

// 3. Konfigurasi Identitas Simpul & Topik
const char* clientId      = "ESP8266_Local_Node01";
const char* topicPubTele  = "kampus/labiot/telemetri";
const char* topicSubCmd   = "kampus/labiot/kendali";

// 4. Alokasi Pin Perangkat Keras
```

Output Serial Monitor ×

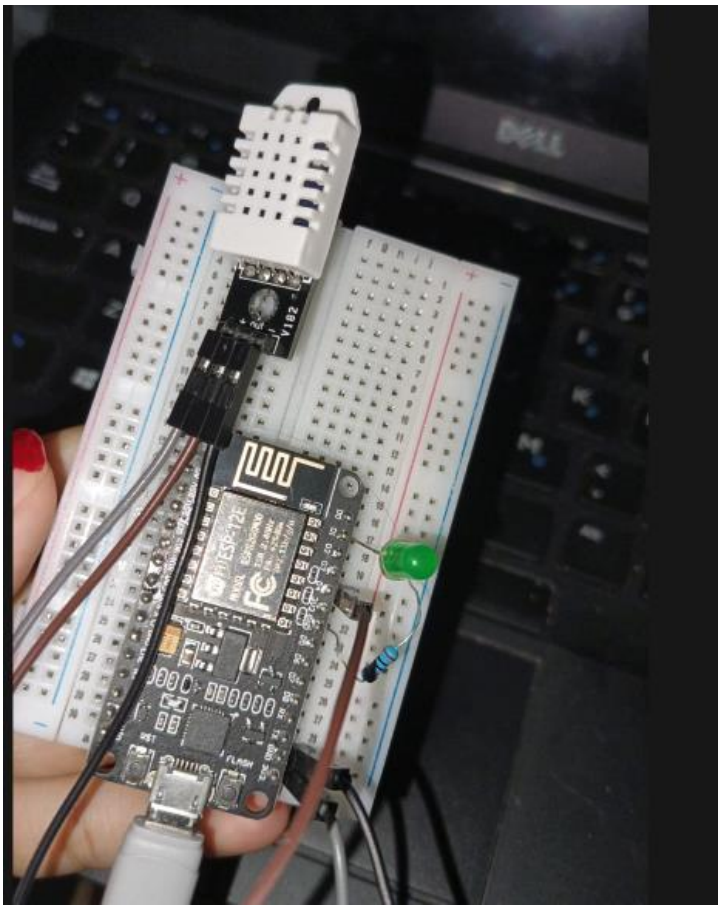
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM6')

New Line 115200 baud

```
----------------------------------------
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":31.10,"kelembapan":40.40,"status_led":1}

----------------------------------------
[INSTRUKSI MASUK] Topik: kampus/labiot/kendali
[PAYLOAD]         : {"led":1}
--> [AKTUATOR] Status LED diubah: MENYALA (ON)
----------------------------------------
```

Ln 12, Col 25 NodeMCU 1.0 (ESP-12E Module) on COM6 2



7.6 Latihan

- **Latihan 1 (Analisis Jaringan Internal Docker & Service Discovery):**
  Pada Praktikum 3, di dalam konfigurasi node mqtt in pada Node-RED, kita mengisi alamat server broker dengan nama teks mqtt, bukan alamat IP lokal laptop (192.168.1.15) maupun localhost.

  i. Mengapa penulisan localhost pada Node-RED di dalam kontainer Docker akan menyebabkan kegagalan koneksi (*Connection Refused*)?

  Penyebab localhost Gagal: Di dalam lingkungan kontainerisasi, localhost (atau IP 127.0.0.1) mengacu pada antarmuka *loopback* internal milik kontainer node-red itu sendiri, **bukan** komputer laptop (*Host OS*) maupun kontainer mqtt-broker. Karena broker Mosquitto berjalan di kontainer terpisah yang memiliki *network namespace* berbeda dan tidak membuka port 1883 di *loopback* node-red, koneksi akan langsung ditolak (*Connection Refused*).

  ii. Jelaskan bagaimana mekanisme *Docker Embedded DNS Server* dan *Bridge Network* bekerja sehingga kontainer node-red dapat menemukan kontainer mqtt-broker cukup dengan memanggil nama layanannya!

  Mekanisme Docker Embedded DNS & Bridge Network: Saat beberapa servis didefinisikan dalam satu berkas docker-compose.yml, Docker secara otomatis membuat sebuah *User-Defined Bridge Network*. Docker menjalankan server DNS internal di alamat IP 127.0.0.11. Ketika node-red meminta koneksi ke *hostname* mqtt (nama nama servis di Compose), server DNS internal Docker akan menerjemahkan nama mqtt tersebut menjadi alamat IP internal kontainer mqtt-broker secara dinamis.

- **Latihan 2 (Penanganan Masalah Konektivitas Jaringan & Subnet Firewall):**
  Seorang mahasiswa mencoba menghubungkan NodeMCU ESP8266 ke broker lokal di laptopnya. ESP8266 berhasil terhubung ke Wi-Fi dan mendapatkan alamat IP 192.168.43.102, sedangkan laptop memiliki alamat IP 192.168.1.15.

  i. Mengapa NodeMCU tidak akan pernah berhasil memublikasikan data ke broker Mosquitto pada kondisi tersebut?

  Penyebab Gagal Komunikasi: Laptop dan ESP8266 berada di subnet/IP kisaran yang berbeda:

Laptop: 192.168.1.15 (Subnet Prefix: 192.168.1.0/24)

ESP8266: 192.168.43.102 (Subnet Prefix: 192.168.43.0/24)

Tanpa adanya perangkat perute (*router*) yang mengaktifkan *Inter-VLAN Routing* atau *IP Forwarding* di antara kedua jaringan ini, paket data paket IP dari ESP8266 tidak akan pernah terkirim ke IP laptop.

ii. Langkah teknis apa yang harus dilakukan mahasiswa tersebut agar kedua perangkat berada dalam domain *broadcast / subnet* yang sama?

- **Menyamakan Jaringan (Satu Wi-Fi / Hotspot yang sama)**: Sambungkan laptop dan ESP8266 ke sumber Wi-Fi/Tethering ponsel yang sama sehingga keduanya mendapatkan IP dari kisaran subnet yang sama (misalnya sama-sama di `192.168.43.x`).

- **Pengaturan Firewall Laptop**: Pastikan *Windows Defender Firewall* (atau firewall OS induk) mengizinkan lalu lintas masuk (*Inbound Rule*) pada **Port TCP 1883**.

- **Latihan 3 (Implementasi Filter Histeresis pada Rule Engine):**
  Pada sistem kendali dua posisi (*Bang-Bang*), jika suhu berfluktuasi cepat di sekitar ambang batas 31.0∘C (misal: 31.05∘C→30.95∘C→31.02∘C), lampu LED indikator akan berkedip-kedip cepat (*flickering / hunting*) yang mengganggu stabilitas visual sistem.
  Rancang modifikasi alur Node-RED menggunakan function node berbahasa JavaScript yang menerapkan logika Histeresis:
  - LED baru menyala jika suhu >31.5∘C (msg.payload = {"led": 1}).
  - LED baru padam jika suhu mendingin hingga <30.0∘C (msg.payload = {"led": 0}).
    Tuliskan kode JavaScript yang Anda masukkan ke dalam *Function Node* tersebut!

```ino
// Callback saat ada instruksi kendali masuk dari Node-RED (via Broker Lokal)
void onMessageCallback(char* topic, byte* payload, unsigned int length) {
  char incomingBuffer[length + 1];
  memcpy(incomingBuffer, payload, length);
  incomingBuffer[length] = '\0';

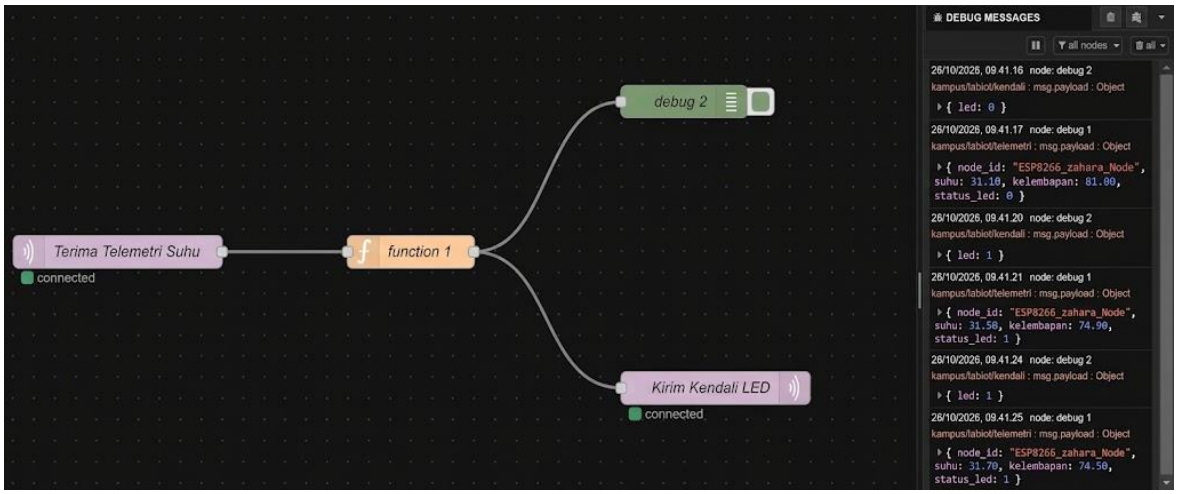
  Serial.println("\n----------------------------------------");
  Serial.printf("[INSTRUKSI MASUK] Topik: %s\n", topic);
  Serial.printf("[PAYLOAD]         : %s\n", incomingBuffer);

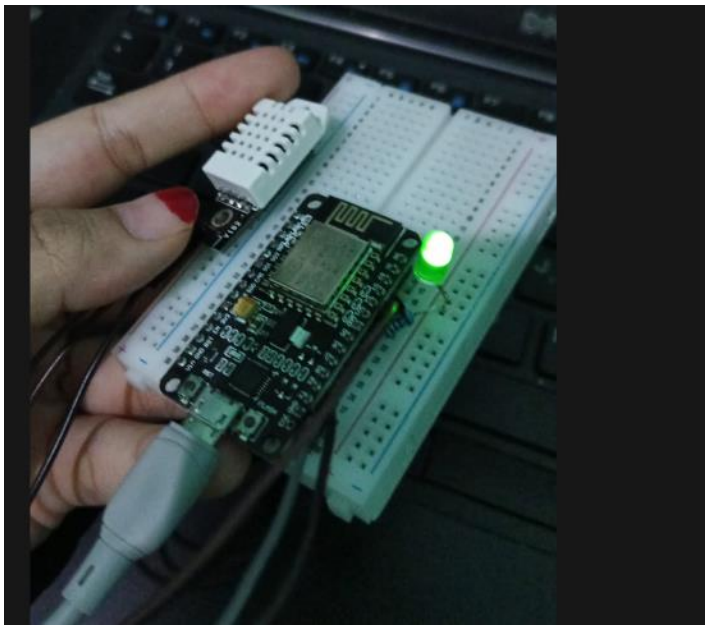
  // Parsing JSON Instruksi Kendali dari Histeresis Node-RED: {"led": 1} atau {"led": 0}
  StaticJsonDocument<256> doc;
  DeserializationError err = deserializeJson(doc, incomingBuffer);

  if (err) {
    Serial.printf("[ERROR] Gagal parsing JSON instruksi: %s\n", err.c_str());
    return;
  }

  // Kendalikan LED fisik berdasarkan hasil keputusan Rule Engine di Node-RED
  if (doc.containsKey("led")) {
    int stateLED = doc["led"];
    digitalWrite(ledPin, stateLED == 1 ? HIGH : LOW);
    Serial.printf("--> [AKTUATOR] Status LED diubah: %s\n", stateLED == 1 ? "MENYALA (ON)" : "PADAM (OFF)");
  } else {
    Serial.println("[WARNING] Kunci 'led' tidak ditemukan dalam payload!");
  }
  Serial.println("----------------------------------------");
}
```







- **Latihan 4 (Analisis Mekanisme Volume Mount & Persistensi Data):**
  Pada berkas docker-compose.yml, terdapat pemetaan direktori - ./nodered:/data dan - ./mosquitto/data:/mosquitto/data.

  i. Apa yang akan terjadi pada alur kanvas visual Node-RED yang telah Anda buat jika kontainer dimatikan dengan perintah docker compose down, kemudian dinyalakan kembali, seandainya baris volume tersebut dihapus dari docker-compose.yml?
  Dampak Penghapusan Volume Mount: Jika baris volumes dihapus dari docker-compose.yml lalu kontainer dimatikan dengan docker compose down, seluruh alur (*flow*), node, serta data Mosquitto yang telah dibuat di kanvas Node-RED akan **hilang secara permanen**. Saat kontainer dinyalakan kembali, Node-RED akan kembali ke kondisi kosong (*default*)

  ii. Jelaskan perbedaan mendasar antara *Bind Mount* dan *Named Volume* pada Docker!
  Bind Mount (./nodered:/data): Memetakan direktori atau berkas yang ada di *Host OS* (komputer laptop) secara spesifik berdasarkan jalur direktori (*path*) lokal. Berkas dapat dilihat dan diedit langsung melalui File Explorer laptop.
  Named Volume (nodered_data:/data): Dikelola sepenuhnya oleh Docker di dalam area penyimpanan internal Docker (/var/lib/docker/volumes/). Lebih disarankan untuk kinerja media penyimpanan performa tinggi dan portabilitas lintas platform OS.

7.7 Tugas

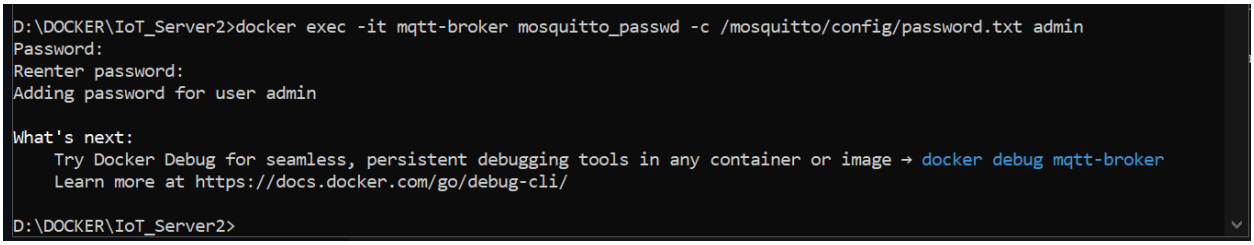
Pengamanan Server IoT Industri (MQTT Authentication & Access Control)

1. Membuat Berkas Kredensial Terenkripsi di Dalam Kontainer:

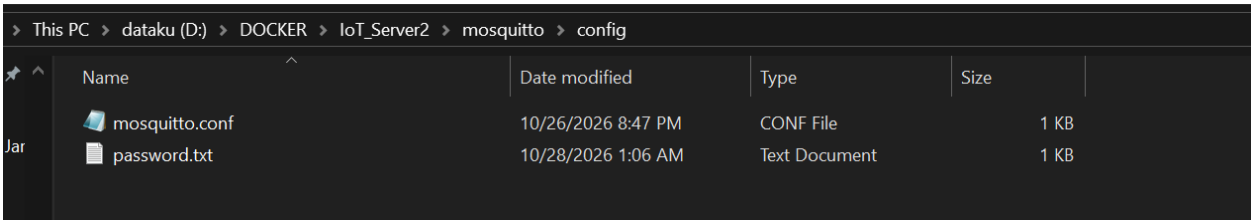
Buka terminal/PowerShell di laptop Anda, lalu jalankan perintah berikut untuk mengeksekusi utilitas mosquitto_passwd langsung di dalam kontainer Mosquitto yang sedang berjalan:

docker exec -it mqtt-broker mosquitto_passwd -c /mosquitto/config/password.txt admin

- Sistem akan meminta Anda memasukkan kata sandi. Masukkan kata sandi: rahasia123 (teks tidak akan muncul di layar saat diketik), lalu tekan Enter dan konfirmasi ulang kata sandi tersebut.



- Buka folder iot-server/mosquitto/config/ pada laptop Anda, pastikan kini terdapat berkas baru bernama password.txt yang berisi baris *hash* kredensial akun admin.



2. Memperbarui Konfigurasi Kebijakan Mosquitto:

Buka berkas iot-server/mosquitto/config/mosquitto.conf di editor teks Anda:

```
protocol websockets

# Perizinan Akses Awal (Tanpa Password)
allow_anonymous true
```

- Ubah nilai allow_anonymous true menjadi:

  allow_anonymous false

- Tambahkan baris pemanggilan berkas kata sandi:

  password_file /mosquitto/config/password.txt

- Simpan berkas mosquitto.conf.
- ```
  # Perizinan Akses Awal (Tanpa Password)
  allow_anonymous false
  password_file /mosquitto/config/password.txt
  ```

3. Memuat Ulang Layanan Broker:

Restart kontainer mqtt-broker melalui terminal agar kebijakan keamanan baru diterapkan:

docker restart mqtt-broker

```
D:\DOCKER\IoT_Server2>docker restart mqtt-broker
mqtt-broker

D:\DOCKER\IoT_Server2>
```

4. Analisis Dampak Kegagalan Sistem (Observasi Masalah):

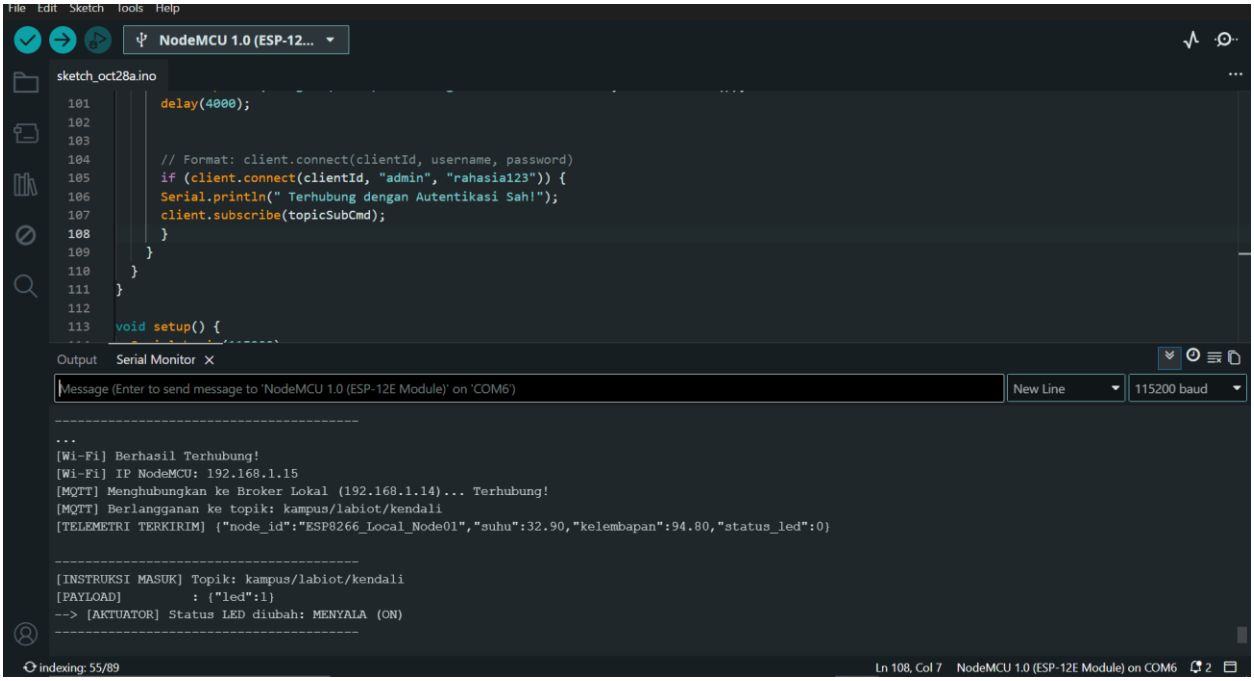
- Buka antarmuka Node-RED di browser: Amati bahwa di bawah node mqtt in dan mqtt out, indikator status kini berubah menjadi warna merah bertuliskan disconnected atau connection refused.
- Buka Serial Monitor NodeMCU: Amati bahwa koneksi mikrokontroler kini gagal dan berulang kali menampilkan kode penolakan:

  [MQTT] Menghubungkan ke Broker Lokal... Gagal (rc=5). Coba lagi dalam 4 detik...

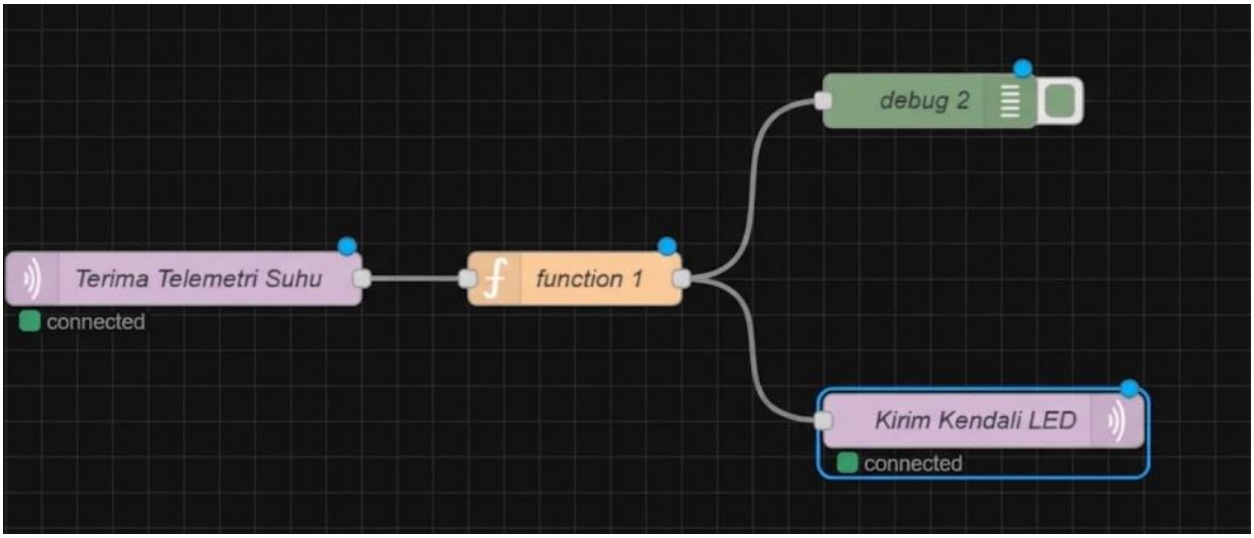
```
........
[Wi-Fi] Berhasil Terhubung!
[Wi-Fi] IP NodeMCU: 192.168.1.15
[MQTT] Menghubungkan ke Broker Lokal (192.168.1.14)... Gagal (rc=5). Coba lagi dalam 4 detik...
[MQTT] Menghubungkan ke Broker Lokal (192.168.1.14)... Gagal (rc=5). Coba lagi dalam 4 detik...
[MQTT] Menghubungkan ke Broker Lokal (192.168.1.14)... Gagal (rc=5). Coba lagi dalam 4 detik...
[MQTT] Menghubungkan ke Broker Lokal (192.168.1.14)... Gagal (rc=5). Coba lagi dalam 4 detik...
[MQTT] Menghubungkan ke Broker Lokal (192.168.1.14)... Gagal (rc=5). Coba lagi dalam 4 detik...
[MQTT] Menghubungkan ke Broker Lokal (192.168.1.14)... Gagal (rc=-2). Coba lagi dalam 4 detik...
[MQTT] Menghubungkan ke Broker Lokal (192.168.1.14)... Gagal (rc=5). Coba lagi dalam 4 detik...
[MQTT] Menghubungkan ke Broker Lokal (192.168.1.14)... Gagal (rc=5). Coba lagi dalam 4 detik...
[MQTT] Menghubungkan ke Broker Lokal (192.168.1.14)... Gagal (rc=5). Coba lagi dalam 4 detik...
[MQTT] Menghubungkan ke Broker Lokal (192.168.1.14)... Gagal (rc=5). Coba lagi dalam 4 detik...
[MQTT] Menghubungkan ke Broker Lokal (192.168.1.14)... Gagal (rc=5). Coba lagi dalam 4 detik...
```
```

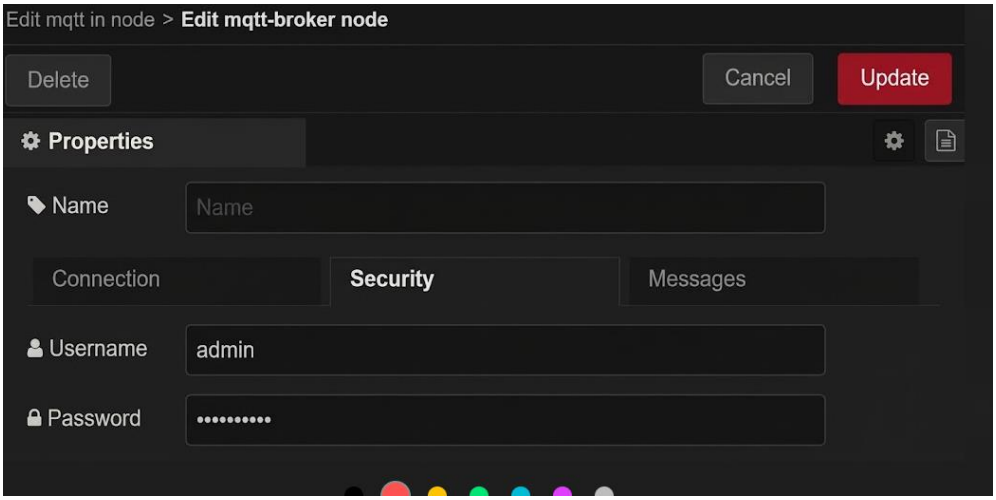
5. Rekayasa Perbaikan Sistem (Penyelesaian Tugas):

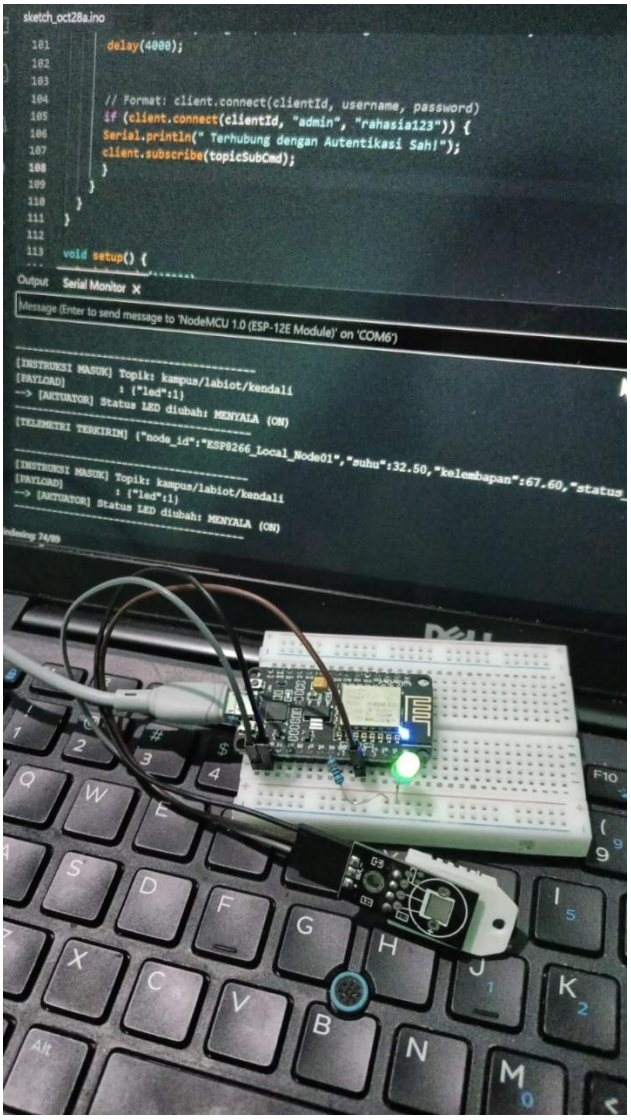
1. Pemutakhiran Skrip Program NodeMCU ESP8266:



2. Pemutakhiran Konfigurasi Kredensial pada Node-RED




Edit mqtt in node > Edit mqtt-broker node
Delete
Cancel
Update
Properties
Name
Name
Connection
Security
Messages
Username
admin
Password
••••••••••


sketch_oct28a.ino
delay(4000);
// Format: client.connect(clientId, username, password)
if (client.connect(clientId, "admin", "rahasia123")) {
Serial.println(" Terhubung dengan Autentikasi Sah!");
client.subscribe(topicSubCmd);
}
}
}
}
void setup() {
Output
Serial Monitor
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM6')
[INSTRUKSI MASUK] Topik: kampus/labiot/kendali
[PAYLOAD] : {"led":1}
--> [AKTUATOR] Status LED diubah: MENYALA (ON)
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":32.50,"kelembapan":67.60,"status
[INSTRUKSI MASUK] Topik: kampus/labiot/kendali
[PAYLOAD] : {"led":1}
--> [AKTUATOR] Status LED diubah: MENYALA (ON)